CS19541 – COMPUTER NETWORKS LAB
PRACTICAL 12

AIM

a) To implement an echo client–server application using TCP/UDP sockets.

b) To implement a chat client–server application using TCP/UDP sockets.

SOFTWARE REQUIRED

Python 3.x and Command Prompt/Terminal. Two terminal windows are required so that the server and client can run separately.

ALGORITHM

1. Import the required **socket** module in the Python program.

2. Create an IPv4 socket using **AF_INET** and select the transport protocol.

3. For TCP communication, use **SOCK_STREAM**; for UDP communication, use **SOCK_DGRAM**.

4. The server binds to a host address and port number. A TCP server listens for a request and accepts a client connection.

5. The client connects to the TCP server, or sends a UDP datagram to the server address.

6. In the echo application, the server receives the client message and sends the same data back.

7. In the chat application, the client and server exchange messages until either side types **exit**.

8. Close the connection/socket and display the messages and final output.

PROCEDURE

1. Create and save the echo server and client programs as **echo_server.py** and **echo_client.py**.

2. Open two terminals in the same folder. Start the server first by running **python echo_server.py**.

3. In the second terminal, run **python echo_client.py**, enter a message, and confirm that the server returns the same message.

4. Save the chat programs as **chat_server.py** and **chat_client.py**. Start the server first, then start the client in another terminal.

5. Exchange messages between the terminals. Type **exit** to stop the chat and close the sockets.

WORKING / DESCRIPTION

The server waits for a client request on a specified port. After connection, the client sends text as bytes. The server receives the data, decodes it when needed, and sends a reply. In the chat program, both sides can send messages repeatedly. The programs use the loopback address **127.0.0.1** to test communication on the same computer.

NOTE

The sample programs on the following pages demonstrate TCP sockets. For UDP testing, use **SOCK_DGRAM** with **sendto()** and **recvfrom()** instead of TCP **connect()** and **accept()**.

PRACTICAL 12 – ECHO CLIENT–SERVER

PROGRAM 1: echo_server.py

```
import socket

HOST = "127.0.0.1"
PORT = 5000

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.bind((HOST, PORT))
    server.listen(1)
    print("Echo server waiting for client...")
    conn, addr = server.accept()
    with conn:
        print("Connected by", addr)
        while True:
            data = conn.recv(1024)
            if not data:
                break
            message = data.decode()
            print("Client:", message)
            conn.sendall(data)
            if message.lower() == "exit":
                break
print("Server closed.")
```

PROGRAM 2: echo_client.py

```
import socket

HOST = "127.0.0.1"
PORT = 5000

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client:
    client.connect((HOST, PORT))
    while True:
        message = input("Enter message (exit to stop): ")
        client.sendall(message.encode())
        reply = client.recv(1024).decode()
        print("Echo from server:", reply)
        if message.lower() == "exit":
            break
print("Client closed.")
```

WORKING

The server waits on port 5000, accepts one TCP connection, receives bytes, and sends the same bytes back. The client prints the returned message. Both programs stop when the client sends **exit**.

OUTPUT – SAMPLE TERMINAL RUN

Terminal 1 – Server

```
Echo server waiting for client...
Connected by ('127.0.0.1', 51432)
Client: Hello server
Client: How are you?
Client: exit
Server closed.
```

Terminal 2 – Client

```
Enter message (exit to stop): Hello server
Echo from server: Hello server
Enter message (exit to stop): How are you?
Echo from server: How are you?
Enter message (exit to stop): exit
Echo from server: exit
Client closed.
```

The terminal text above is an illustrative sample; capture your own run for submission if required.

PRACTICAL 12 – CHAT CLIENT–SERVER

PROGRAM 3: chat_server.py

```
import socket

HOST = "127.0.0.1"
PORT = 5001

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.bind((HOST, PORT))
    server.listen(1)
    print("Chat server waiting for client...")
    conn, addr = server.accept()
    with conn:
        print("Connected by", addr)
        while True:
            data = conn.recv(1024)
            if not data:
                break
            message = data.decode()
            print("Client:", message)
            if message.lower() == "exit":
                break
            reply = input("You: ")
            conn.sendall(reply.encode())
            if reply.lower() == "exit":
                break
print("Chat ended.")
```

PROGRAM 4: chat_client.py

```
import socket

HOST = "127.0.0.1"
PORT = 5001

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client:
    client.connect((HOST, PORT))
    while True:
        message = input("You: ")
        client.sendall(message.encode())
        if message.lower() == "exit":
            break
        reply = client.recv(1024).decode()
        print("Server:", reply)
        if reply.lower() == "exit":
            break
print("Chat ended.")
```

OUTPUT – SAMPLE TERMINAL RUN

Terminal 1 – Server

```
Chat server waiting for client...
Connected by ('127.0.0.1', 51440)
Client: Hello
You: Hi!
Client: How are you?
You: I am fine.
Client: exit
Chat ended.
```

Terminal 2 – Client

```
You: Hello
Server: Hi!
You: How are you?
Server: I am fine.
You: exit
Chat ended.
```

RESULT

Thus, TCP socket programs for echo client–server and chat client–server communication were implemented. Messages were exchanged successfully and the received output was verified.

RUNNING THE PROGRAMS

Open two terminals in the program folder. Start **python echo_server.py** before **python echo_client.py**. For chat, start **python chat_server.py** before **python chat_client.py**. Use separate runs for each pair.